

SHACL and Racer nRQL:

What Was Conceptually New?

A semantic comparison of SHACL validation with RacerPro nRQL, the OWL/Data Mirroring Substrate, and hybrid nRQL queries

Prepared from the uploaded RacerPro manuals and the SHACL specifications

Author: Generative AI (OpenAI GPT-5.5 Pro)

See the Generative AI authorship and independence declaration.

June 1, 2026

Abstract

This report compares SHACL with what was already expressible in RacerPro through nRQL, the OWL Data Mirroring Substrate, the substrate representation layer, and hybrid queries. The main conclusion is deliberately narrow: for OWL-centric integrity constraints over finite, explicitly represented data, very little in SHACL is fundamentally new at the level of constraint expressivity. nRQL already provided closed-world violation queries over an open-world OWL knowledge base by combining DL-aware positive atoms with negation as failure, projection, aggregation, rules, concrete-domain/datatype predicates, complex TBox queries, and hybrid substrate atoms. The important correction to many modern comparisons is that nRQL was not merely an ABox query language: the mirror data substrate could expose OWL elements, datatype values, annotation values, and other mirrored objects as queryable substrate data.

In pragmatic terms, the comparison supports the claim that the large majority of ordinary, contemporary SHACL use cases over OWL-centric data—plausibly on the order of four fifths, if one counts recurring patterns rather than deployments—were already expressible with nRQL plus RacerPro’s substrate and mirroring machinery by 2005. This “80 percent” figure should not be read as a measured empirical statistic. It is a compact way of saying that required-property checks, cardinality checks, qualified-value checks, disjointness and equality checks, missing-annotation checks, ontology-metadata checks, closed-world violation queries, and many rule-assisted data-completeness checks were already technically available. The residual cases are the genuinely RDF-native ones: regular property paths over raw graphs, literal language/lexical term primitives, closed-shape inspection of arbitrary predicate positions unless faithfully mirrored, and the shape-conformance/reporting semantics introduced by SHACL itself.

The revised report also separates two historical claims that are often conflated. The priority claim is strong: the supplied RacerPro manual explicitly says that nRQL was among the first expressive DL query languages with negation as failure and projection, and it further claims that nRQL was at that time the only practically available OWL query language for such autoepistemic queries. In the technical sense of NEG applied to projected query bodies and defined query bodies, nRQL had negated-subquery and universal closed-world-query idioms years before SPARQL 1.1 standardized FILTER NOT EXISTS, MINUS, and subqueries. The influence claim is weaker: priority and conceptual anticipation do not prove that SPARQL 1.1, SHACL, or property-graph query languages such as Cypher directly took their ideas from nRQL. The report therefore treats direct influence as undocumented unless explicit citations or design records show it.

The genuinely new or partially new conceptual contributions of SHACL are therefore not the ability to say “a required value is missing” or “an ontology element lacks an annotation.” Those patterns were already expressible in nRQL. The remaining semantic novelties are mostly (i) a first-class shape-conformance relation over RDF graphs, rather than an answer-set query discipline; (ii) RDF term and graph structure as the primitive validation universe; (iii) built-in regular property-path evaluation, especially unbounded path operators, over the raw RDF graph; and (iv) when SHACL-SPARQL is admitted, the imported SPARQL algebra and SPARQL term/function vocabulary. Even these differences often disappear if the relevant RDF term structure or path closures are mirrored or materialized into a RacerPro substrate. Conversely, nRQL was stronger than SHACL Core in its native integration of OWL reasoning with local closed-world querying and rule-based ABox augmentation.

Generative AI authorship and independence declaration

This report was authored by a generative AI system, OpenAI GPT-5.5 Pro. The human user supplied the research question, the uploaded RacerPro manuals, clarifications about Racer/nRQL features, and requests to examine particular historical claims. Those inputs were treated as source material and hypotheses to be assessed, not as conclusions to be adopted automatically.

The analytical findings in this report were not coerced and were not accepted merely because they were suggested, strongly suggested, emphasized, or desired by the user. Where the user proposed

interpretations—for example, the significance of negated projected subqueries, the OWL/Data Mirroring Substrate, property-graph-like substrate encodings, or the approximate “80 percent” headline—the report either checked those interpretations against the available manual evidence and logical comparison, or else marked them as estimates, qualifications, or undocumented influence claims. The report’s distinctions between priority, conceptual anticipation, and direct influence reflect the AI author’s independent judgment based on the evidence available in this conversation.

This declaration should not be read as a claim that the report is archival proof of historical influence. In particular, the report credits Racer/nRQL as strong prior art where supported by the uploaded manuals, but it does not assert documented causal influence on SPARQL 1.1, SHACL, Cypher, or other property-graph query languages unless independent citations or design records establish such influence.

Contents

Generative AI authorship and independence declaration	i
1 Question and scope	1
2 A calibrated headline: the “80%” claim	2
3 Historical priority: negated subqueries and universal closed-world queries	3
3.1 What can be substantiated from the RacerPro sources	3
3.2 Universal closed-world queries	4
3.3 Comparison with SPARQL 1.0 and SPARQL 1.1	4
4 The data substrate and property graphs	5
4.1 Racer substrate graphs as property-graph encodings	5
4.2 Origins of property graphs: priority versus lineage	5
5 Influence on SPARQL 1.1, SHACL, and property-graph query languages	6
5.1 Methodological distinction	6
5.2 SPARQL 1.1	6
5.3 SHACL	6
5.4 Cypher and property-graph query languages	7
5.5 On non-credit and bibliographic omissions	7
6 Semantic baseline: what nRQL already had	7
6.1 Open-world OWL plus local closed-world querying	7
6.2 Positive atoms as entailment, negative atoms as failure to find answers	8
6.3 Rules and ABox augmentation	8
6.4 Concrete domains, datatypes, strings, and aggregation	9
6.5 TBox queries	9
6.6 The substrate representation layer and hybrid queries	9
6.7 The OWL/Data Mirroring Substrate	10
7 A constraint-theoretic view	10
7.1 Violation queries and shape conformance	10
7.2 Informal expressivity theorem	11

8	Translation patterns	11
8.1	Core constraints	11
8.2	Axiom and annotation governance	14
8.3	Closed shapes	14
9	Where SHACL is not fundamentally new	15
9.1	Missing required values	15
9.2	Constraints over annotations and ontology metadata	15
9.3	Local closed-world constraints over an open-world ontology	15
9.4	Qualified cardinality checks over known values	16
9.5	Rule-like data completion	16
10	What is genuinely or conditionally new in SHACL	16
10.1	A first-class conformance relation	16
10.2	RDF graph terms as the primitive universe	17
10.3	Regular property paths over the raw graph	17
10.4	SHACL-SPARQL imports SPARQL algebra and functions	17
10.5	Native literal and language constraint components	18
10.6	Validation reports, severities, and messages	18
11	Where nRQL was stronger than SHACL Core	18
11.1	Native OWL reasoning inside constraints	18
11.2	Hybrid semantic atoms	19
11.3	Non-monotonic update rules	19
12	Boundary cases	19
12.1	Asserted versus entailed validation	19
12.2	Axiom annotations versus entailed consequences	19
12.3	Blank nodes and anonymous OWL existentials	20
13	Answer to the main question	20
14	Conclusion	21
A	Manual evidence summarized	22

1 Question and scope

The question is not whether SHACL became a standard, attracted more implementations, or integrated better with generic RDF tooling. Those are important historical facts, but they are outside the scope requested here. The question is conceptual and semantic:

What did SHACL make expressible as a constraint language that was not already expressible, in principle, in RacerPro by nRQL plus the substrate representation layer, the mirror data substrate, and hybrid nRQL queries?

This report uses a generous interpretation of “possible with nRQL” because that matches the system described in the RacerPro manuals. The comparison includes:

1. nRQL ABox queries with classical negation, negation as failure, projection, union, and conjunction.
2. DL-aware query atoms evaluated by logical entailment over the RacerPro knowledge base.
3. Rules whose antecedents are nRQL query bodies and whose consequences are generalized ABox assertions.
4. Concrete-domain and datatype predicates, including string and numeric predicates.
5. Group-by and aggregation operators, mentioned in the user guide as part of nRQL’s expressive query facilities.
6. Complex TBox queries, treated in the guide as nRQL queries over a taxonomy ABox.
7. Hybrid queries over an ABox plus an associated substrate representation layer.
8. The mirror data substrate, including its use for OWL documents, annotations, and told datatype or annotation values.

The main sources for the RacerPro side are the uploaded *RacerPro User’s Guide Version 2.0* and *RacerPro Reference Manual Version 1.9*. The user guide explicitly describes nRQL as an expressive query language for DL systems with variables over named domain objects, negation as failure, a projection operator, group-by, and aggregation [1, p. 4]. Its summary of nRQL features emphasizes compositional semantics, variables bound by logical entailment, negation as failure and classical negation, concrete-domain constraints, projection, datatype/annotation retrieval, complex TBox queries, lambda expressions, aggregation, rules, and hybrid substrate queries [1, pp. 81–84]. The mirror data substrate is described as automatically creating corresponding substrate objects for objects in an ABox and, for OWL KBs, for elements in the OWL KB; it also exposes annotation and told datatype values to substrate queries [1, pp. 137–143].

The SHACL side is split into SHACL Core, SHACL-SPARQL, and, where relevant, SHACL Advanced Features. SHACL Core is the shape language with standard constraint components such as `sh:minCount`, `sh:maxCount`, `sh:class`, `sh:datatype`, `sh:nodeKind`, `sh:closed`, logical combinators, qualified value shapes, and property paths. SHACL-SPARQL permits constraints written as SPARQL queries. SHACL Advanced Features adds rule-like mechanisms, but it is not needed for most validation examples discussed here.

2 A calibrated headline: the “80%” claim

The compact way to state the result is this:

For the common OWL/RDF integrity-validation repertoire that SHACL is often used for today, a large majority—plausibly on the order of 80%—was already expressible about twenty years ago in RacerPro via nRQL, negation as failure, projection, rules, concrete-domain/datatype predicates, and the substrate/mirror-data representations.

This is intentionally not presented as a measured bibliometric or industrial-usage statistic. Establishing a literal percentage would require a corpus of current SHACL shapes from production systems and a reproducible coding scheme. The point is instead a scoped technical estimate about the usual denominator in SHACL discussions: required properties, missing values, cardinalities, type constraints, datatype and string constraints, qualified value restrictions, disjointness/equality constraints, annotation-governance constraints, ontology metadata checks, and closed-world absence-of-counterexample constraints over finite RDF/OWL data.

Within that denominator, the estimate is strong. The following categories, which account for much of ordinary SHACL practice, were already in reach of 2005-era nRQL plus Racer:

Common SHACL use-case family	Racer/nRQL status
Required value or missing property, e.g. <code>sh:minCount 1</code>	Expressible as NEG over a projected successor query, or by the documented known-successor idiom.
Maximum/minimum cardinality and duplicate-value checks	Expressible through query patterns, inequality, aggregation/grouping where needed, and local closed-world violation queries.
All values of a property must satisfy a type or shape condition	Expressible as absence of projected counterexamples: no known value exists that fails the required condition.
Qualified value constraints	Expressible by projecting away the witness variable and checking existence or non-existence of values satisfying a class, role, datatype, or substrate predicate condition.
Ontology-element and annotation-governance checks	Expressible when OWL classes, properties, axioms, annotations, told datatype values, and annotation values are mirrored into the data substrate.
Datatype, numeric, string, and concrete-domain checks	Supported in the Racer/nRQL setting to the extent documented concrete-domain and substrate predicates cover the required operations.
Rule-derived validation predicates and repair/augmentation patterns	Supported through nRQL rules whose antecedents are query bodies and whose consequents add generalized ABox assertions.

The residual roughly “20%” should be read as the part of SHACL practice that is genuinely more RDF-native or SPARQL-specific than the usual OWL-centric integrity-constraint fragment. Examples include native RDF term inspection over raw graph nodes, language-tag-specific constraints such as unique-language validation unless language tags are mirrored as queryable values, closed-shape checks over arbitrary predicate positions unless the substrate exposes triples or predicates as first-class query objects, unbounded regular property paths over raw RDF graphs, and SHACL-SPARQL constraints whose substance depends on SPARQL-specific algebra, term functions, or property-path machinery.

So the intended claim is neither that SHACL added nothing nor that every possible SHACL-SPARQL program had a direct nRQL equivalent. The intended claim is that SHACL’s famous practical niche—closed-world validation of OWL/RDF data—was substantially prefigured and implemented in Racer/nRQL. The genuinely new part is smaller and more precise: RDF-native shape conformance, built-in validation-result semantics, native graph-term/path machinery, and SPARQL-specific facilities when SHACL-SPARQL is used.

3 Historical priority: negated subqueries and universal closed-world queries

3.1 What can be substantiated from the RacerPro sources

The strongest historically relevant statement in the supplied manual occurs in the early overview of nRQL. The guide says that nRQL was “one of the first expressive query languages for DL systems” and immediately lists conjunctive queries, variables over named domain objects, negation as failure, a projection operator, group-by, and aggregation operators [1, p. 4]. A few pages later, in the feature summary, it makes a still stronger claim for the OWL setting: nRQL is described as the only practically available OWL query language at that time supporting execution of such “autoepistemic” queries, namely queries that ask for what is known and what is not known in the KB [1, p. 82].

The manual also documents the exact technical idiom that matters for the comparison with later SPARQL and SHACL practice. nRQL query bodies are built compositionally with **and**, **union**, **neg**, and **project-to** [1, pp. 81–82]. The **NEG** constructor implements negation as failure [1, pp. 113–117]. The **project-to** operator can be placed inside a query body, not only in the final answer head, and the guide explicitly explains that the correct way to ask for individuals with no known *R*-successor satisfying a condition is to apply **NEG** after the internal projection [1, pp. 119–120]:

Listing 1: nRQL negated projected query body, i.e. an anti-existential subquery.

```
(retrieve (?x)
  (neg (project-to (?x)
    (and (?x c) (?x ?y r) (?y d))))))
```

This is, in modern database terminology, an anti-semi-join or **NOT EXISTS** pattern over a projected subquery. In addition, nRQL defined queries can be used wherever a query body is accepted and can themselves be negation as failure-negated [1, pp. 120–122]:

Listing 2: nRQL negation of a defined query body.

```
(defquery mother-of (?x ?y)
  (and (?x woman) (?x ?y has-child)))

(retrieve (?a ?b)
  (neg (substitute (mother-of ?a ?b))))
```

Thus, if the phrase “negated subquery” is understood semantically rather than in the exact SPARQL surface syntax, nRQL plainly had it: a query expression could contain a projected or named query body and could apply negation as failure to that body. This is the mechanism needed for universal closed-world queries.

3.2 Universal closed-world queries

Universal constraints over a finite active domain are normally reduced to absence of counterexamples. For a fixed focus object x ,

$$\forall y (P(x, y) \rightarrow Q(y))$$

can be checked by asking whether there is no y such that $P(x, y)$ and not $Q(y)$:

$$\neg \exists y (P(x, y) \wedge \neg Q(y)).$$

Under closed-world query semantics, this becomes a projected failure query. The query that returns conforming focus objects has an outer NEG over the projected counterexample set:

Listing 3: Closed-world universal condition by absence of counterexamples.

```
(retrieve (?x)
  (and (?x FocusClass)
    (neg
      (project-to (?x)
        (and (?x ?y P)
          (neg (?y Q)))))))
```

The corresponding violation query simply returns the projected counterexample focus objects:

Listing 4: Violation query for a failed closed-world universal condition.

```
(retrieve (?x)
  (and (?x FocusClass)
    (project-to (?x)
      (and (?x ?y P)
        (neg (?y Q))))))
```

The same shape appears in SHACL as a universal value constraint, for example “all values of path P conform to shape Q ”. SHACL expresses it as a shape-conformance condition; nRQL can express both the conforming objects and the violating objects by choosing whether the projected counterexample body is negated. At the conceptual level, the universal closed-world query pattern was therefore already present in nRQL.

3.3 Comparison with SPARQL 1.0 and SPARQL 1.1

The relevant dates support the priority claim. The RacerPro bibliography lists *Querying the Semantic Web with Racer + nRQL* in 2004 and the formal nRQL semantics paper *A high performance semantic web query answering engine* in 2005 [1, p. 213,215]. SPARQL 1.0 became a W3C Recommendation in 2008 [5]; SPARQL 1.1 became a W3C Recommendation in 2013 [10]. SPARQL 1.1 is the version that standardized general negation constructs such as FILTER NOT EXISTS and MINUS, along with subqueries. SPARQL 1.0 had OPTIONAL plus FILTER(!BOUND(...)), which could simulate many simple negation-as-failure idioms, but it did not have the general SPARQL 1.1 negation and subquery apparatus.

Therefore the cautious, well-supported statement is this:

By the documented 2004–2005 nRQL publications, and certainly by the user-guide semantics, nRQL already supported negated projected query bodies and negated defined query bodies for RDF/OWL/DL querying. This predates SPARQL 1.1’s standardized `FILTER NOT EXISTS`, `MINUS`, and subquery features by several years. The supplied manual even characterizes nRQL as the only practically available OWL query language of its time for such autoepistemic queries.

The stronger statement, “nRQL was globally the first OWL/Semantic Web query language with negated subqueries,” is plausible in the narrower OWL/DL-query sense, but it is a historical priority claim that ideally requires an exhaustive comparison with every contemporary RDF query language, rule language, and research prototype. The manuals support “one of the first” and “the only practically available OWL query language” for the autoepistemic feature; they do not by themselves prove global firstness over every experimental system.

4 The data substrate and property graphs

4.1 Racer substrate graphs as property-graph encodings

The RacerPro substrate layer is also relevant to property graphs. The user guide defines a data substrate as a node- and edge-labeled directed graph whose labels may be built from Common Lisp symbols, strings, numbers, and characters [1, p. 133]. It then describes hybrid queries in which some atoms address the ABox and other atoms address the associated substrate [1, pp. 133–137]. The mirror data substrate creates corresponding substrate data objects for ABox objects and, for OWL KBs, for OWL elements; it also exposes annotation relationships, annotation values, datatype values, and datatype roles as queryable substrate data [1, pp. 137–143].

A contemporary property graph is normally described as a directed graph whose vertices and edges can carry labels and key-value properties. A Racer data substrate is not identical to every modern property-graph data model: in particular, modern property graphs usually treat edges as first-class entities with their own property maps. Still, the substrate can encode property graphs straightforwardly. Vertices become substrate nodes; graph edges become substrate edges; vertex properties become node labels, attached data nodes, or substrate predicates; and edge properties can be represented as edge labels, attached value nodes, or reified edge objects. Once encoded, hybrid nRQL queries can combine property-graph-style traversal and attribute filtering with DL reasoning over the mirrored or associated ABox.

This means that property-graph-style data modeling was not foreign to the RacerPro/nRQL architecture. The substrate component was already a finite labeled graph model with graph predicates and hybrid links to OWL reasoning.

4.2 Origins of property graphs: priority versus lineage

The broader property-graph lineage is not a single clean invention point. It descends from older attributed and labeled graph models, semantic networks, object-oriented graph database models, and graph database systems. The graph-database survey literature before the current property-graph wave already documented many graph data models and query languages [7]. The specific term “property graph” became prominent in the Neo4j/TinkerPop/Blueprints ecosystem around the late 2000s and early 2010s, with later formal and industrial descriptions of property-graph query languages such as Cypher [8, 9].

Against that background, the fairest statement is not that nRQL or Racer invented property graphs. The fair statement is narrower but still important:

Racer’s substrate layer provided a labeled directed graph representation, graph predicates, hybrid graph-plus-DL queries, and an OWL mirroring mechanism. This gave nRQL a way to encode and query property-graph-like structures before the current property-graph ecosystem became dominant, and at least contemporaneously with the early public emergence of the modern property-graph terminology.

Again, this is a priority/anticipation claim, not a documented direct-lineage claim.

5 Influence on SPARQL 1.1, SHACL, and property-graph query languages

5.1 Methodological distinction

Historical comparison must distinguish three different claims:

1. **Prior art:** nRQL had a feature before a later language standardized or popularized a similar feature.
2. **Conceptual convergence:** two communities independently developed similar mechanisms because they faced the same finite graph/query/constraint problem.
3. **Direct influence:** designers of the later language knew of, used, cited, or intentionally built on nRQL or Racer.

The first claim is well supported for negated projected queries and local closed-world OWL querying. The second claim is highly plausible. The third claim requires documentary evidence such as citations, design notes, acknowledgments, mailing-list discussions, or shared implementation histories. In the absence of such evidence, direct influence should not be asserted.

5.2 SPARQL 1.1

SPARQL 1.1’s `FILTER NOT EXISTS`, `MINUS`, and subqueries are conceptually close to nRQL’s `NEG` over projected query bodies, at least for the finite graph-query fragment. But SPARQL’s documented lineage is primarily the RDF query-language and database-query lineage, including earlier RDF query systems and SQL-style constructs. The SPARQL specifications themselves do not present these features as inherited from nRQL.

The strongest fair conclusion is therefore:

nRQL anticipated, in an OWL/DL-aware Semantic Web query setting, the negated-subquery patterns that SPARQL 1.1 later made standard for RDF graph querying. I do not have documentary evidence that SPARQL 1.1 took direct inspiration from nRQL.

5.3 SHACL

SHACL’s closed-world validation style overlaps heavily with nRQL violation-query idioms. However, the documented SHACL design lineage is more visibly connected to RDF validation, SPARQL constraints, SPIN, OSLC Resource Shapes, and Shape Expressions than to DL query languages [6, 3]. This does not diminish the technical priority of nRQL for OWL-centric closed-world queries; it means that direct influence is not established by the visible citation trail.

The accurate claim is:

Many of SHACL’s practical use cases had already been expressible in nRQL plus Racer’s mirror/substrate machinery. SHACL should therefore be viewed less as the invention of closed-world constraints for OWL data and more as an RDF-native shape-conformance formulation of a constraint idea that nRQL had already operationalized for OWL/DL KBs.

5.4 Cypher and property-graph query languages

Cypher and related property-graph query languages address graph-pattern matching, traversal, and property filtering over labeled property graphs. nRQL’s hybrid substrate queries address a related problem: graph-pattern and predicate querying over a labeled substrate, optionally coupled to DL reasoning. There is genuine conceptual overlap. But current property-graph query languages seem to have emerged from graph-database and application-development needs, not from the OWL/DL reasoner-query tradition.

Thus the fair conclusion is:

nRQL and Racer’s substrate machinery should be recognized as prior Semantic Web/DL technology capable of encoding property-graph-like structures and querying them in hybrid form. I do not have evidence that Cypher or other property-graph languages directly borrowed from nRQL.

5.5 On non-credit and bibliographic omissions

If later Semantic Web validation or graph-query literature presents closed-world constraints, negated graph patterns, or universal constraint queries as if they emerged only with SPARQL 1.1 or SHACL, that history is incomplete. Racer/nRQL deserves to be mentioned as prior art for OWL-aware autoepistemic querying, negated projected query bodies, and hybrid graph-plus-DL querying. The absence of citations in later standards and industrial language documents is best described as a bibliographic and disciplinary gap unless direct knowledge and deliberate non-attribution can be shown.

6 Semantic baseline: what nRQL already had

6.1 Open-world OWL plus local closed-world querying

OWL axioms are interpreted under open-world semantics. An existential restriction can imply that an individual has some filler without requiring a named, explicitly asserted filler in the ABox. Therefore an OWL axiom of the form “every parent has some child” is not a data-completeness constraint.

nRQL already addressed this gap. The RacerPro guide explicitly distinguishes classical negation from negation as failure and uses negation as failure for measuring the completeness of modeling in an ABox [1, pp. 113–116]. The guide’s own example is very close to the familiar SHACL use case: retrieve people without known, explicitly modeled children. It explains why a naive negated binary role atom gives the wrong answer after projection, then gives the correct idiom:

Listing 5: nRQL idiom for missing known successors.

```
(retrieve (?x)
  (neg (project-to (?x) (?x ?y has-child))))
```

The manual also states that this is equivalent to using `has-known-successor` [1, pp. 115–116, 119–120]. Thus the central SHACL pattern

Listing 6: SHACL idiom for the same missing-successor constraint.

```
ex:ParentShape
  a sh:NodeShape ;
  sh:targetClass ex:Parent ;
  sh:property [
    sh:path ex:hasChild ;
    sh:minCount 1
  ] .
```

has a direct nRQL counterpart:

Listing 7: nRQL violation query for the parent example.

```
(retrieve (?x)
  (and (?x Parent)
    (neg (project-to (?x) (?x ?y has-child))))))
```

This is not a marginal feature. It is precisely the combination of active-domain bindings, projection, and negation as failure that makes local closed-world constraints possible while retaining DL reasoning for positive atoms.

6.2 Positive atoms as entailment, negative atoms as failure to find answers

A central semantic distinction is that nRQL positive ABox atoms are not merely syntactic triple checks. The user guide says that a variable is bound to an ABox individual only if the individual satisfies the query, and that satisfaction means the query after substitution is logically entailed by the KB [1, p. 82]. Thus an nRQL atom such as

```
(?x Parent)
(?x ?y hasChild)
```

can exploit RacerPro’s DL reasoning, depending on the selected reasoning mode. At the same time, `NEG` can be used to compute a complement relative to the active domain of known individuals or substrate nodes. This combination gives nRQL a hybrid epistemic profile: open-world entailment for positive DL atoms, closed-world failure for selected query subexpressions.

This profile is already close to how SHACL is often used over OWL data: shapes validate a finite graph under a closed-world reading, while the graph may be the asserted graph, an RDFS/OWL-materialized graph, or a graph supplied by an entailment-aware processor. The difference is that RacerPro made the DL-entailment side native, whereas SHACL requires an explicit entailment/materialization choice outside the shape language.

6.3 Rules and ABox augmentation

nRQL also had a rule mechanism. The user guide describes rules as an ABox augmentation mechanism; rule antecedents are ordinary nRQL query bodies and rule consequences are generalized ABox assertions [1, pp. 125–127]. The reference manual states that `firerule` is the rule equivalent of `retrieve`, that there are no TBox rules, and that non-monotonic rules can be written because of negation as failure [2, pp. 160–161].

For example, the guide gives a rule that creates a new child for each known mother lacking a known child:

Listing 8: ABox augmentation with a missing-successor antecedent.

```
(firerule
  (and (?x mother)
        (neg (?x (has-known-successor has-child))))
  ((instance (new-ind first-child-of ?x) human)
   (related (new-ind first-child-of ?x) ?x has-mother)))
```

This is more than validation. It is update-producing, possibly non-monotonic ABox augmentation. SHACL Core does not provide this. SHACL Advanced Features has rules, but the conceptual pattern of query antecedent plus generated assertions was already present in RacerPro.

6.4 Concrete domains, datatypes, strings, and aggregation

The user guide emphasizes that nRQL supports complex retrieval conditions over concrete-domain attribute fillers and OWL datatype-property fillers [1, pp. 82–83]. It also mentions group-by and aggregation operators in the historical overview of nRQL [1, p. 4] and later describes lambda expressions that can implement efficient aggregation operators such as `sum` and `avg` [1, p. 83]. The substrate language additionally supports data predicates over symbols, strings, numbers, and characters, including string search and numeric comparison [1, pp. 136–137, 155–156].

This matters because several SHACL constraints are often presented as novel because they are not OWL axioms: value range checks, string checks, datatype checks, and value comparisons. But these were not foreign to nRQL. They were already expressible either on the concrete-domain side, the OWL datatype side, or the substrate side.

6.5 TBox queries

The user guide describes complex TBox queries and states that nRQL TBox queries are essentially nRQL ABox queries posed to a specific “taxonomy ABox” constructed from the TBox taxonomy [1, p. 132]. Therefore constraints over classes and subclass patterns were not intrinsically outside nRQL.

This matters for shape-like governance rules such as:

Every class below a given superclass must have a label, definition, provenance annotation, or axiom annotation of a specified kind.

Without the substrate/mirroring context, one might incorrectly classify such constraints as “metadata validation” that nRQL could not express. With TBox queries and mirroring, that classification is too strong.

6.6 The substrate representation layer and hybrid queries

The substrate representation layer is the key feature for a fair comparison. The user guide says that a substrate can be associated with an ABox to form a hybrid representation; nRQL is then a hybrid query language in which some atoms address the ABox and other atoms address the substrate [1, pp. 133–137]. The guide explicitly motivates this by saying that expressive means not allowed in the ABox part for decidability reasons can be offered in the substrate part for constraint and model checking [1, pp. 83–84].

A data substrate is a node- and edge-labeled directed graph whose labels are built from Common Lisp symbols, strings, numbers, and characters [1, p. 133]. Substrate variables are distinct from ABox variables but can be bound in parallel with corresponding ABox objects [1, pp. 135–136].

Data predicates can be used in node and edge query atoms; the guide explicitly shows substring search and binary numeric comparison, including comparisons between nodes even where no explicit edge exists [1, pp. 136–137].

This is already very close to a finite graph validation language. It also explains why the right comparison is not “SHACL versus OWL” or even “SHACL versus DL queries,” but “SHACL versus DL-aware finite graph model checking plus negation as failure and rules.”

6.7 The OWL/Data Mirroring Substrate

The mirror data substrate is decisive. The guide describes it as automatically creating a corresponding, appropriately labeled substrate data object for every object in a given ABox; when an OWL file has been read, mirroring creates data substrate objects for elements in the OWL KB [1, p. 137]. It creates nodes for ABox individuals, concrete-domain objects, and concrete-domain values, and edges for related and constrained assertions [1, pp. 138–139]. It adds markers for ABox individuals, concrete-domain values, role relationships, attribute relationships, told-value relationships, and, in the OWL case, markers such as `:owl-annotation`, `:owl-annotation-relationship`, `:owl-annotation-value`, `:owl-datatype-value`, and `:owl-datatype-role` [1, pp. 140–141].

The manual then explains that, for OWL KBs, the mirror data substrate is valuable because OWL KBs often contain annotations and told XML Schema datatype values; with the mirror one can query resources whose annotation-property fillers contain a substring, and variables can be bound to told datatype or annotation fillers because these told values are accessible as substrate nodes [1, pp. 141–143].

Therefore the following claim would be false:

SHACL can validate ontology annotations, classes, properties, and axiom metadata,
whereas nRQL was limited to ordinary domain individuals.

The manuals show that nRQL plus the mirror substrate could query such structures as data. It was therefore possible to express constraints such as “every subclass axiom of a given kind must carry an annotation of a required kind,” assuming the relevant axiom/annotation representation is mirrored or otherwise represented in the substrate.

7 A constraint-theoretic view

7.1 Violation queries and shape conformance

Most integrity constraints can be expressed in one of two equivalent ways:

1. as a conformance predicate, saying that a focus node satisfies a shape; or
2. as a violation query, returning focus nodes and values that do not satisfy the constraint.

SHACL chooses the first presentation as the primary abstraction and produces validation results for non-conformances. nRQL naturally chooses the second presentation: a constraint is a query whose answer set should be empty, or whose answers are the violations to be reported.

For example, the SHACL condition

Every `ex:Parent` has at least one `ex:hasChild` value.

has the nRQL violation query already shown:

```
(retrieve (?x)
  (and (?x Parent)
    (neg (project-to (?x) (?x ?y hasChild))))))
```

The conceptual gap between these presentations is small. A validation report is a structured rendering of violation bindings. Conversely, every nRQL violation query can be read as defining a shape-like non-conformance condition.

7.2 Informal expressivity theorem

The following theorem is not a claim about syntax or implementation convenience. It states the main semantic point of the comparison.

Informal theorem. Assume an RDF/OWL graph G is represented in a RacerPro knowledge base together with a faithful substrate mirror $M(G)$ that exposes the RDF terms, triples or role edges, literal values, annotation values, and any required term descriptors as substrate nodes, edges, and labels. For every nonrecursive SHACL Core constraint whose property paths are either bounded path expressions or have been materialized as finite binary relations, there is an nRQL violation query that returns exactly the nonconforming focus/value bindings, modulo the chosen entailment regime.

Proof sketch. Targets become the outer retrieval domain: `sh:targetClass C` becomes a concept atom or a mirrored type query; `sh:targetSubjectsOf p` and `sh:targetObjectsOf p` become substrate edge queries. Property paths become joins over role atoms or substrate edges, or calls to materialized path relations. `sh:minCount 1` becomes NEG after PROJECT-TO; higher lower bounds become either existence of n distinct value variables or aggregation. `sh:maxCount n` becomes existence of $n + 1$ pairwise distinct values, or an aggregate count violation. Value-class, datatype, node-kind, language, pattern, and enumeration constraints become concept atoms, concrete-domain/datatype atoms, or substrate label/predicate checks. Logical shape combinators map to AND, UNION, and NEG. Qualified value shapes map to projected existence or counting over values satisfying the nested query body. Closed shapes map to a query for an outgoing mirrored edge whose predicate label is not among the allowed property labels. The construction is by structural induction over the nonrecursive shape expression. $\square$

The theorem intentionally assumes a faithful substrate representation. That assumption is justified for the OWL-centric cases discussed in the manuals: RacerPro’s mirror substrate already exposed OWL annotations and told datatype/annotation values as queryable data [1, pp. 141–143]. For arbitrary non-OWL RDF data, an application may need to create an explicit data substrate representation.

8 Translation patterns

8.1 Core constraints

Table 2 summarizes common SHACL Core patterns and their nRQL counterparts. It does not claim that the translations are syntactically complete for all corner cases; rather, it shows that the constraint pattern was already semantically available.

Table 2: SHACL Core patterns and nRQL equivalents.

SHACL pattern	Semantic reading	nRQL/Racer counterpart
<code>sh:targetClass C</code>	Validate nodes known or treated as instances of C .	Concept atom $(?x\ C)$ if using Racer entailment; mirrored <code>rdf:type</code> query if validating raw graph data.
<code>sh:targetSubjectsOf p</code>	Focus nodes with outgoing p .	Role/substrate edge atom $(?x\ ?y\ p)$ or mirrored edge query.
<code>sh:minCount 1</code>	There exists at least one value along the path.	Violation: $(\text{neg}(\text{project-to}(\text{?x})\ \text{path-body}))$; exactly the manual’s missing known successor idiom.
<code>sh:minCount n</code>	At least n distinct values.	Existence of n pairwise distinct value variables, or count/group-by aggregation.
<code>sh:maxCount n</code>	No more than n distinct values.	Violation: existence of $n + 1$ pairwise distinct values; use injective variables or explicit <code>same-as</code> /inequality constraints.
<code>sh:class C</code>	Each value conforms to class C .	Violation: path value v plus $(\text{neg}(\text{?v}\ C))$; use DL entailment or raw mirrored type/subclass data depending on desired semantics.
<code>sh:datatype d</code>	Each value is a literal of datatype d .	Concrete-domain/datatype query atoms, or mirrored literal-value/type markers in the substrate.
<code>sh:nodeKind</code>	IRI/blank/literal term-kind restriction.	Substrate labels or markers for term kind, if represented. Not an OWL DL fact, but easily a finite-data predicate.
<code>sh:pattern</code>	String lexical check.	Concrete-domain string predicates where available; substrate predicates/lambda functions for string checks. The manual explicitly shows substring search in substrate queries.

SHACL pattern	Semantic reading	nRQL/Racer counterpart
<code>sh:languageIn,</code> <code>sh:uniqueLang</code>	Language-tag constraints over literal terms.	Substrate representation of language tags plus ordinary value comparison/counting. Native only if language tags are mirrored as data.
<code>sh:hasValue, sh:in</code>	Required or enumerated values.	Equality/same-as or substrate literal equality; violation via negation as failure if required value is absent.
<code>sh:equals,</code> <code>sh:disjoint</code>	Compare value sets of two paths.	Difference queries using projection and negation as failure; disjointness as existence of a shared value.
<code>sh:lessThan,</code> <code>sh:lessThanOrEquals</code>	Cross-property value comparison.	Concrete-domain or substrate binary predicates such as numeric/string comparison.
<code>sh:qualifiedMinCount,</code> <code>sh:qualifiedMaxCount</code>	Count values satisfying a nested shape.	Project/count values satisfying the nested query body; use NEG for missing qualified values.
<code>sh:and, sh:or, sh:not,</code> <code>sh:xone</code>	Boolean combinations of conformance tests.	<code>and</code> , <code>union</code> , <code>neg</code> ; exclusive-or via Boolean combinations and counting of satisfied alternatives.
<code>sh:closed true</code>	No outgoing properties except allowed properties.	Query for a mirrored outgoing relationship edge whose predicate label is not in the allowed set; requires predicate labels to be represented in the substrate.
<code>sh:path</code> sequence/inverse/alternative	Navigate finite graph paths.	Joins with intermediate variables; inverse roles or inverse substrate edges; union for alternatives.
<code>sh:zeroOrMorePath,</code> <code>sh:oneOrMorePath</code>	Regular-path reachability.	Native only if the path relation is a transitive role, recursive rule closure, or precomputed substrate relation. This is one of the few real semantic pressure points.

8.2 Axiom and annotation governance

Because of the mirror data substrate, nRQL could query OWL annotations and told values as data. Therefore an SHACL-SPARQL shape such as the following is not conceptually beyond nRQL:

Listing 9: SHACL-SPARQL shape for required annotations on subclass axioms.

```
ex:AnnotatedSubClassAxiomShape
  a sh:NodeShape ;
  sh:targetClass owl:Axiom ;
  sh:sparql [
    sh:message "Subclass axioms must have a required annotation." ;
    sh:select """
      SELECT $this
      WHERE {
        $this owl:annotatedSource ?x ;
              owl:annotatedProperty rdfs:subClassOf ;
              owl:annotatedTarget ?y .
        FILTER NOT EXISTS {
          $this ?p ?v .
          ?p a owl:AnnotationProperty .
          ?p ex:annotationKind ex:RequiredKind .
        }
      }
    """
  ] .
```

A schematic hybrid nRQL violation query has the same structure:

Listing 10: Schematic nRQL version over a mirrored OWL substrate.

```
(retrieve (?*ax ?*x ?*y)
  (and (?*ax ?*x (:owl-annotated-source))
    (?*ax *rdfs:subClassOf (:owl-annotated-property))
    (?*ax ?*y (:owl-annotated-target))
    (neg
      (project-to (?*ax)
        (and (?*ax ?*p (:owl-annotation-relationship))
          (?*p RequiredAnnotationKind))))))
```

The precise marker names depend on the substrate encoding. The point is semantic: once axiom nodes, sources, targets, properties, and annotations are mirrored as data, this is an ordinary finite graph constraint with negation as failure and projection.

8.3 Closed shapes

A closed SHACL shape is often considered graph-native:

```
ex:PersonShape
  a sh:NodeShape ;
  sh:targetClass ex:Person ;
  sh:closed true ;
  sh:ignoredProperties ( rdf:type ) ;
  sh:property [ sh:path ex:name ; sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path ex:email ; sh:maxCount 1 ] .
```

The violation condition is:

There exists an outgoing property edge from the focus node whose predicate is not `rdf:type`, `ex:name`, or `ex:email`.

If the substrate represents outgoing RDF triples or role edges with predicate labels, this becomes an nRQL query for an existing relationship edge plus negation as failure against the allowed predicate alternatives:

```
(retrieve (?*x ?*y)
  (and (?x Person)
    (?*x ?*y (:abox-relationship))
    (neg
      (union (?*x ?*y (:predicate (equalp rdf:type)))
        (?*x ?*y (:predicate (equalp ex:name)))
        (?*x ?*y (:predicate (equalp ex:email)))))))
```

This is not meant as exact Racer syntax for arbitrary URI symbols; it is the shape of the query. The manual’s data-substrate language supports node/edge labels and predicates over labels [1, pp. 133,136–137,155–156]. Thus the semantic requirement is a faithful edge-label representation, not a fundamentally new validation logic.

9 Where SHACL is not fundamentally new

9.1 Missing required values

This is the clearest non-novelty. SHACL’s `sh:minCount 1` is a direct validation construct, but the underlying semantics is exactly the kind of projected failure query that nRQL documented in its missing known successor example. nRQL even had syntactic sugar for `has-known-successor` [1, pp. 115–116].

9.2 Constraints over annotations and ontology metadata

This is the point most easily missed. Once the OWL Data Mirroring Substrate is included, nRQL can query OWL annotation values, datatype values, and other mirrored OWL objects as substrate data. The guide explicitly states that this is valuable for OWL KBs with annotations and told datatype values and that users can query annotation-property fillers with substring predicates [1, pp. 141–143]. Therefore SHACL did not uniquely enable ontology-governance constraints such as required labels, definitions, provenance, or axiom annotations.

9.3 Local closed-world constraints over an open-world ontology

The combination of OWL semantics and closed-world data constraints is often described as a reason to use SHACL. But nRQL had already made this combination explicit: positive atoms could be DL-entailment-aware, while `NEG` and projection supported failure tests over the active domain. The user guide itself states that negation as failure is useful for measuring modeling completeness and calls such queries “autoepistemic” [1, p. 82].

9.4 Qualified cardinality checks over known values

SHACL qualified value shapes, such as “at least one child that is a Child,” translate naturally:

```
ex:ParentShape
  a sh:NodeShape ;
  sh:targetClass ex:Parent ;
  sh:property [
    sh:path ex:hasChild ;
    sh:qualifiedValueShape [ sh:class ex:Child ] ;
    sh:qualifiedMinCount 1
  ] .
```

```
(retrieve (?p)
  (and (?p Parent)
    (neg
      (project-to (?p)
        (and (?p ?c hasChild)
          (?c Child))))))
```

This uses the same projection-after-existential-hiding pattern that the manual discusses in detail for `project-to` [1, pp. 119–120].

9.5 Rule-like data completion

SHACL Core is not a rule language. SHACL Advanced Features adds rules, but nRQL rules already had query-body antecedents, generalized ABox consequences, creation of new individuals, and non-monotonic behavior through negation as failure [1, pp. 125–127]. Therefore rule-like augmentation is not a conceptual novelty of SHACL relative to nRQL; if anything, nRQL was stronger than SHACL Core here.

10 What is genuinely or conditionally new in SHACL

10.1 A first-class conformance relation

The first genuine conceptual novelty is not expressive power in the narrow query-theoretic sense. It is the choice of a first-class validation semantics:

A data graph G conforms to a shapes graph S when all focus nodes satisfy the applicable shapes, and violations are structured as validation results.

nRQL could express the same conditions as violation queries, but “being a shape” and “conforming to a shape” were not its primary semantic objects. In nRQL, the primary object is an answer set for a query or a set of generated rule consequences. In SHACL, the primary object is conformance/non-conformance of focus nodes to shapes.

This matters conceptually because SHACL defines validation as a relation between a data graph and a shapes graph, not as a side convention over arbitrary queries. However, this is a semantic organization of validation, not a strong increase in what constraints can be stated.

10.2 RDF graph terms as the primitive universe

SHACL validates RDF graphs directly. Its primitive universe includes IRIs, blank nodes, literals, datatype IRIs, language tags, and triples. nRQL's core ABox variables range over named individuals; substrate variables range over substrate nodes. The mirror data substrate closes much of this gap for OWL KBs, and a user-created data substrate can close more of it. Nevertheless, RDF-term-level validation is native in SHACL and represented in nRQL only to the extent that the term structure is mirrored.

This distinction is semantic when a constraint depends on term features that RacerPro did not mirror natively. Examples include:

1. whether a value is a blank node rather than an IRI;
2. the exact lexical form of a literal;
3. a language tag and not merely the string value;
4. syntactic features of RDF collections or reification structures;
5. duplicate graph terms that become semantically identical under OWL reasoning.

If those features are encoded as substrate labels, nRQL can query them. If they are not encoded, SHACL sees them and nRQL does not. Thus this is a conditional novelty: native in SHACL, dependent on mirroring in nRQL.

10.3 Regular property paths over the raw graph

SHACL property paths inherit a SPARQL-like path language with sequences, inverses, alternatives, and unbounded operators such as zero-or-more and one-or-more paths. Bounded sequences, inverses, and alternatives translate well to nRQL joins, inverse roles, and unions. The hard case is unbounded reachability over an arbitrary predicate in the raw RDF graph:

```
[ sh:path [ sh:zeroOrMorePath ex:broader ] ; sh:minCount 1 ]
```

This is a regular path query. Plain nonrecursive relational algebra with difference and projection does not define arbitrary transitive closure over finite graphs. nRQL can recover the behavior if the property is declared transitive in the OWL/DL layer, if a recursive rule/fixpoint computation materializes the closure, or if the closure is precomputed into the substrate. But as a primitive graph-navigation construct independent of OWL role declarations, `sh:zeroOrMorePath` and `sh:oneOrMorePath` are among the strongest candidates for a genuine semantic addition.

The qualification is important. If the relevant closure is materialized, or if the role semantics already supplies transitivity, then nRQL can query it. What SHACL adds is direct regular-path validation over the RDF graph.

10.4 SHACL-SPARQL imports SPARQL algebra and functions

SHACL-SPARQL permits validators written as SPARQL queries. To the extent that those validators rely on SPARQL-specific algebra or functions, they may go beyond native nRQL syntax. Examples include SPARQL property paths, subqueries, `OPTIONAL`, `MINUS`, `VALUES`, `BIND`, RDF term functions such as `isBlank`, `isLiteral`, `datatype`, `lang`, and the SPARQL expression/function library.

However, this should not be overstated. Many SPARQL patterns used in SHACL constraints are just joins, filters, projections, and `FILTER NOT EXISTS`; those correspond closely to nRQL

conjunction, substrate predicates, projection, and negation as failure. The manual also notes termination-safe lambda expressions and server-side programming facilities, including aggregation [1, p. 83]. Therefore the gap is not “SPARQL constraints versus no constraints.” It is SPARQL’s particular RDF algebra and function vocabulary versus nRQL’s DL/substrate/lambda vocabulary.

10.5 Native literal and language constraint components

SHACL Core has named components such as `sh:languageIn`, `sh:uniqueLang`, `sh:pattern`, and `sh:datatype`. These are semantically convenient because they are defined directly over RDF literal terms.

For nRQL, datatype values and annotation values could be retrieved through OWL datatype-/annotation facilities and the mirror data substrate, and string search was available in substrate predicates [1, pp. 141–143, 155–156]. But the manuals do not establish that every RDF literal detail, especially language-tag structure, was always exposed natively. If language tags and lexical forms are mirrored into the substrate, nRQL can express the constraints. If not, SHACL has native semantic access that nRQL lacks.

10.6 Validation reports, severities, and messages

SHACL defines structured validation results with focus nodes, value nodes, result paths, messages, severities, and source shapes. nRQL returns answer tuples or rule consequences. A wrapper could convert nRQL answers into a report, but the report vocabulary and result-object semantics are built into SHACL.

Because the requested comparison excludes standards and popularity, this should not be treated as a decisive semantic expressivity gain. Still, it is a conceptual difference: SHACL makes diagnostic reporting part of the validation model, while nRQL leaves reporting as an application convention over answer sets.

11 Where nRQL was stronger than SHACL Core

11.1 Native OWL reasoning inside constraints

nRQL’s positive atoms are evaluated against the RacerPro KB and can exploit DL entailment. Thus a constraint can mix:

1. entailed class membership;
2. entailed role membership;
3. classical DL negation;
4. negation as failure over projected answer sets;
5. substrate model checking over mirrored data;
6. concrete-domain and datatype predicates.

SHACL Core, by contrast, validates a data graph. Entailment may be supplied by preprocessing or by a processor’s entailment regime, but it is not intrinsic to most shape expressions. Therefore nRQL was often semantically stronger for OWL-aware constraints.

11.2 Hybrid semantic atoms

Hybrid nRQL queries can bind ABox variables and substrate variables in parallel, allowing a single query to combine semantic OWL conditions with graph/data-substrate predicates [1, pp. 135–137]. This is more integrated than the typical SHACL Core setup, where graph validation and reasoning are separated by an entailment/materialization boundary.

11.3 Non-monotonic update rules

nRQL rules could add ABox assertions and could be non-monotonic because their antecedents could use negation as failure [1, pp. 125–127]. SHACL Core has no update semantics. SHACL Advanced Features has rules, but this is not a conceptual capability that postdates nRQL in the broad sense relevant to this report.

12 Boundary cases

12.1 Asserted versus entailed validation

Both systems can be used to validate asserted data or entailed data, but they do so differently. In nRQL, the chosen query mode and mirror mode determine how much told versus inferred information participates. The user guide says, for example, that mirroring can use only told syntactic information in one mode, add atomic concept ancestors and synonyms in another, and add further complex concept information in a smarter mode [1, pp. 139–140]. In SHACL, the analogous choice is usually made by deciding whether to validate the raw data graph, an entailed graph, or a materialized graph.

Thus the systems answer different questions unless configured carefully:

- Raw SHACL: Does the triple occur in the validation graph?
- Entailment-aware SHACL: Does the triple or class relation occur after the selected entailment regime?
- nRQL over the ABox: Is the atom entailed by the RacerPro KB?
- nRQL over the mirror substrate: Is the told or smart-mirrored substrate pattern present?

No semantic comparison is meaningful without fixing this axis.

12.2 Axiom annotations versus entailed consequences

A constraint such as

Every asserted subclass axiom must have a provenance annotation.

is a finite graph/axiom-object constraint and is expressible in both systems if axiom annotations are represented.

A different constraint is:

Every entailed subclass relation $C \sqsubseteq D$ must have a provenance annotation.

This is not a normal annotation constraint, because an entailed relation may not correspond to a single asserted axiom object. It may follow from a proof or chain of axioms. nRQL can query the entailed subclass relation through TBox reasoning; SHACL can see it only if materialized. But attaching an annotation to the consequence requires a provenance or explanation model in either system. This is not solved by SHACL merely by being graph-native.

12.3 Blank nodes and anonymous OWL existentials

SHACL validates blank nodes as graph terms if they occur in the data graph. OWL existential consequences may introduce anonymous model elements that are not graph terms and not named ABox individuals. nRQL variables in the documented ABox language range over named ABox individuals or mirrored substrate nodes, not over arbitrary model witnesses. This distinction explains why **has-known-successor** is so important: it asks for known successors, not merely existentially implied ones.

13 Answer to the main question

Under the broad RacerPro setting described in the manuals, the answer is:

SHACL did not fundamentally introduce the ability to express closed-world integrity constraints over OWL data, required-property constraints, qualified cardinality constraints, annotation-governance constraints, or many datatype/string checks. These were already expressible in nRQL using negation as failure, projection, rules, concrete-domain/datatype predicates, and the mirror/substrate representation layer.

What SHACL did introduce at a conceptual/semantic level is narrower:

1. **Not negated subqueries or universal closed-world querying.** Those were already present in nRQL through NEG plus PROJECT-T0, and the manuals explicitly connect this to autoepistemic OWL querying.
2. **A shape-conformance semantics.** Constraints are first-class shapes whose satisfaction by focus nodes is the core semantic relation. nRQL can encode the same violations, but as queries.
3. **A native RDF graph validation universe.** IRIs, blank nodes, literals, language tags, and triples are primitive validation objects. nRQL can match this only insofar as those objects are mirrored into a substrate.
4. **Native regular property paths.** Bounded paths translate to nRQL joins; unbounded graph reachability is native in SHACL paths but requires role transitivity, recursive rule closure, or materialization in nRQL.
5. **SPARQL algebra and term functions when SHACL-SPARQL is used.** This imports capabilities that are not identical to nRQL's native query language, especially RDF term functions and SPARQL path/algebra constructs. Many common uses, however, still translate to hybrid nRQL.
6. **Built-in diagnostic result objects.** This is semantically about validation reporting, not about the expressibility of the underlying constraints.

If one excludes standardization, popularity, and tooling, and if one grants RacerPro a faithful substrate representation of the relevant RDF graph and term structure, the remaining fundamental expressivity advantage of SHACL Core is quite small. The most defensible hard gap is native regular-path validation over raw RDF graphs. The most important soft gap is that SHACL takes RDF graph terms and validation reports as primitives, whereas nRQL requires a query/substrate encoding and an application convention for reporting. In that scoped sense, the informal “80%” summary is a fair executive headline: most ordinary SHACL use cases that amount to finite, closed-world OWL/RDF integrity checks were already supportable in nRQL plus Racer; the remaining minority is concentrated in RDF-native term/path semantics, SHACL’s shape-conformance/reporting model, and SPARQL-specific machinery.

14 Conclusion

A historically fair comparison should not say that SHACL was the first practical way to impose closed-world constraints on OWL data, nor that SPARQL 1.1 was the first Semantic-Web setting in which negated subqueries made universal closed-world checks possible. RacerPro’s nRQL already supplied the essential mechanisms: negation as failure, projection, logical-entailment-aware query atoms, rules, datatype and concrete-domain predicates, group/aggregation, complex TBox queries, and a hybrid substrate layer. The mirror data substrate is especially important because it made OWL annotations and told values queryable as finite data, eliminating a common but incorrect distinction between “domain ABox constraints” and “ontology metadata constraints.”

The central semantic difference is instead one of orientation. nRQL is a powerful reasoner-integrated query and rule language that can be used to implement validation. SHACL is a graph validation language whose semantic objects are shapes, focus nodes, value nodes, paths, and validation results. That orientation gives SHACL native RDF-term validation and property-path validation, while nRQL gives stronger native integration with OWL reasoning and non-monotonic query/rule idioms.

Therefore, the answer to “what was fundamentally new?” is not “constraints” and not “closed-world checks.” It is primarily the RDF-native shape-conformance framework, plus a few graph-specific primitives such as regular paths and native term/literal validation. Most ordinary SHACL constraints over OWL ontologies, including constraints over classes, properties, subclass axioms, annotations, and explicitly known role fillers, were already within reach of nRQL plus RacerPro’s substrate machinery.

As a deliberately approximate summary, one can say that perhaps 80 percent of the recurring practical SHACL validation patterns seen in OWL/RDF ontology-engineering work were already supportable in RacerPro by 2005, provided the comparison includes nRQL, rules, negated projected subqueries, concrete-domain predicates, aggregation, hybrid substrate queries, and the OWL/Data Mirroring Substrate. The percentage is not a survey result; it is a shorthand for the feature-by-feature conclusion of this report. The remaining 20 percent is not unimportant, but it is different in kind: it consists mainly of RDF-native term inspection, regular graph-path validation, exact raw-graph closed-shape validation unless represented in the substrate, and SHACL’s own first-class conformance/report vocabulary.

The revised historical conclusion is also that nRQL deserves explicit recognition as prior art for negated subqueries in an OWL/Semantic Web query language. The supplied manual documents NEG over projected query bodies and defined queries, identifies negation as failure as a mechanism for autoepistemic OWL querying, and presents nRQL as uniquely practical for that purpose at the time. This predates SPARQL 1.1’s general negation and subquery facilities. What remains unproven is

direct influence: the conceptual debt is clear at the level of prior art and technical anticipation, but not as a documented citation lineage from nRQL to SPARQL 1.1, SHACL, or Cypher.

A Manual evidence summarized

The following source points from the uploaded manuals are most relevant to the comparison.

1. The user guide describes nRQL historically as an expressive DL query language with conjunctive queries, variables over named domain objects, negation as failure, projection, group-by, and aggregation [1, p. 4].
2. The feature summary states that nRQL atoms can be combined with **and**, **union**, **neg**, and **project-to**; variables bind to ABox individuals satisfying the query by logical entailment; negation as failure and classical negation are both available; concrete-domain constraints and datatype/annotation retrieval are supported; projection and aggregation are available; and hybrid substrate queries are supported [1, pp. 81–84].
3. The guide’s negation as failure section explains the difference between classical negation and negation as failure and gives the missing-known-child pattern using **project-to** and **has-known-successor** [1, pp. 113–120].
4. The rules section defines nRQL rules as ABox augmentation with ordinary nRQL query bodies as antecedents and generalized ABox assertions as consequences; it shows non-monotonic behavior and creation of new individuals [1, pp. 125–127]. The reference manual confirms that **firerule** is the rule counterpart of **retrieve** and that negation as failure permits non-monotonic rules [2, pp. 160–161].
5. The substrate section defines a substrate as a node- and edge-labeled directed graph associated with an ABox, and hybrid nRQL queries as queries containing both ABox and substrate atoms [1, pp. 133–137].
6. The mirror data substrate automatically creates substrate data objects for ABox objects and, for OWL KBs, for elements in the OWL KB; it includes markers for OWL annotations, annotation relationships, annotation values, datatype values, and datatype roles [1, pp. 137–141].
7. The guide explicitly states that, with the mirror data substrate, users can query OWL annotation-property fillers with substring predicates and bind variables to told datatype or annotation fillers because those values are accessible as substrate nodes [1, pp. 141–143].
8. The guide says that nRQL was one of the first expressive DL query languages and lists negation as failure, projection, group-by, and aggregation as early features [1, p. 4]. It later describes nRQL as the only practically available OWL query language of its time for autoepistemic queries [1, p. 82].
9. The guide shows **NEG** applied to **project-to** and to defined query bodies, which is the key negated-subquery idiom for closed-world universal constraints [1, pp. 119–122].

References

- [1] Racer Systems GmbH & Co. KG. *RacerPro User’s Guide Version 2.0*. October 24, 2012. Uploaded as `users-guide-2-0.pdf`.

- [2] Racer Systems GmbH & Co. KG. *RacerPro Reference Manual Version 1.9*. October 12, 2010. Uploaded as `reference-manual.pdf`.
- [3] Holger Knublauch and Dimitris Kontokostas, editors. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 20 July 2017. <https://www.w3.org/TR/shacl/>.
- [4] Holger Knublauch, Dimitris Kontokostas, and others. *SHACL Advanced Features*. W3C Note, 8 June 2017. <https://www.w3.org/TR/shacl-af/>.
- [5] Eric Prud'hommeaux and Andy Seaborne, editors. *SPARQL Query Language for RDF*. W3C Recommendation, 15 January 2008. <https://www.w3.org/TR/rdf-sparql-query/>.
- [6] Holger Knublauch. *SPIN - Modeling Vocabulary*. W3C Member Submission, 22 February 2011. <https://www.w3.org/Submission/spin-modeling/>.
- [7] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1), Article 1, 2008.
- [8] Marko A. Rodriguez and Peter Neubauer. Constructions from dots and lines. *Bulletin of the American Society for Information Science and Technology*, 36(6):35–41, 2010.
- [9] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andres Taylor. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018.
- [10] Steve Harris and Andy Seaborne, editors. *SPARQL 1.1 Query Language*. W3C Recommendation, 21 March 2013. <https://www.w3.org/TR/sparql11-query/>.
- [11] Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau, editors. *OWL 2 Web Ontology Language: Direct Semantics*. W3C Recommendation, 11 December 2012. <https://www.w3.org/TR/owl2-direct-semantics/>.